Leena.app v401 - JWT + Expo Backend Ozeti

1. .env dosyasi

```
PGUSER=postgres
PGHOST=localhost
PGDATABASE=leena_v401
PGPASSWORD=
PGPORT=5432
JWT_SECRET=supersecureleena
```

2. db.js

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  user: process.env.PGUSER,
  host: process.env.PGHOST,
  database: process.env.PGDATABASE,
  password: process.env.PGPASSWORD,
  port: process.env.PGPORT,
});

module.exports = pool;
```

3. auth.js

```
const express = require('express');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
const router = express.Router();
const pool = require('../utils/db');
require('dotenv').config();

router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  try {
    const result = await pool.query('SELECT * FROM organizers WHERE email = $1', [email]);

    if (result.rows.length === 0) return res.status(401).json({ error: 'Invalid email or password'
});

    const organizer = result.rows[0];
    const match = await bcrypt.compare(password, organizer.password_hash);
    if (!match) return res.status(401).json({ error: 'Invalid email or password' });

    if (!process.env.JWT_SECRET) throw new Error('JWT_SECRET not defined in .env');

          const token = jwt.sign({ organizer_id: organizer.id, email: organizer.email },
process.env.JWT_SECRET, { expiresIn: '8h' });
```

```
    res.json({ token });
  } catch (err) {
    console.error('Login error:', err);
    res.status(500).json({ error: 'Internal server error' });
  }
});

module.exports = router;
```

4. authMiddleware.js

```
const jwt = require('jsonwebtoken');
const pool = require('../utils/db');
require('dotenv').config();

const authenticateToken = async (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Token missing' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.organizer_id = decoded.organizer_id;

    // await pool.query('SET LOCAL leena.organizer_id = $1', [decoded.organizer_id]); // RLS icin
hazir

    next();
  } catch (err) {
    console.error('Token auth error:', err);
    res.status(403).json({ error: 'Invalid token' });
  }
};

module.exports = authenticateToken;
```

5. expos.js

```
const express = require('express');
const router = express.Router();
const pool = require('../utils/db');
const authenticateToken = require('../middleware/authMiddleware');

router.get('/', authenticateToken, async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM expos WHERE organizer_id = $1 ORDER BY id',
[req.organizer_id]);
    res.json(result.rows);
  } catch (err) {
    console.error('Error fetching expos:', err);
    res.status(500).json({ error: 'Internal server error' });
  }
});
```

```
router.post('/', authenticateToken, async (req, res) => {
  const { name, location, start_date, end_date, logo_url } = req.body;
  if (!name || !start_date || !end_date) {
    return res.status(400).json({ error: 'Missing required fields' });
  }

  try {
    const result = await pool.query(
      `INSERT INTO expos (organizer_id, name, location, start_date, end_date, logo_url)
       VALUES ($1, $2, $3, $4, $5, $6) RETURNING *`,
      [req.organizer_id, name, location, start_date, end_date, logo_url]
    );
    res.status(201).json(result.rows[0]);
  } catch (err) {
    console.error('Error adding expo:', err);
    res.status(500).json({ error: 'Internal server error' });
  }
});

module.exports = router;
```